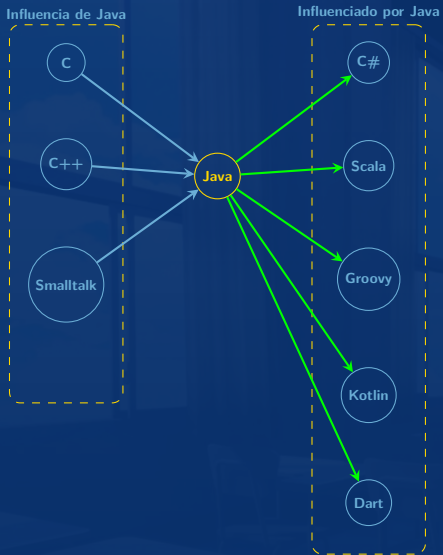




Universidad Autónoma de Baja California
Facultad de Ciencias Químicas e Ingeniería
ISTE

LPOO

Unidad 0
Repaso Estructurada



Java es un lenguaje de programación orientado a objetos que fue desarrollado originalmente por un equipo de ingenieros de **Sun Microsystems**, liderados por ****James Gosling****, a principios de la década de 1990.

A finales de los 80 e inicios de los 90, Sun Microsystems buscaba nuevas formas de innovar y anticiparse a la tendencia de la “computación distribuida” y los dispositivos inteligentes.

Se formó un grupo de trabajo, conocido como el proyecto “Green”, en el que participaban James Gosling, Mike Sheridan, Patrick Naughton, entre otros.

En 1991, James Gosling comenzó a desarrollar un lenguaje llamado inicialmente “Oak”, en referencia a un roble (oak tree) que veía desde su ventana.

La idea original de Oak era que fuera un lenguaje portátil y eficiente para dispositivos embebidos (por ejemplo, decodificadores de TV por cable, dispositivos domésticos inteligentes, etc.).

Sin embargo, el mercado de la tecnología de consumo y los dispositivos digitales aún no estaba listo para un lenguaje así.

Oak se renombró después como Java por motivos de marca y registro (además de que “Oak” ya estaba utilizado por otro software).

La principal innovación de Java fue su lema **“Write once, run anywhere”** (WORA), que se logró gracias a la Java Virtual Machine (JVM) y a los applets, pequeños programas que podían correr en navegadores compatibles sin depender de la arquitectura del sistema.

Sun Microsystems lanzó oficialmente Java 1.0 en 1995, con mucha publicidad enfocada en su capacidad multiplataforma, ya que el mismo bytecode podía ejecutarse en diferentes sistemas operativos.

- **Portabilidad:** “Escribir una vez, ejecutar en cualquier parte” (WORA), gracias a la JVM.
- **Orientación a objetos:** Un modelo de clases, interfaces y herencia simple y efectivo.
- **Gestión automática de memoria** (garbage collector): Facilita la programación y reduce errores de gestión manual de memoria.
- **Seguridad y robustez:** El bytecode se ejecuta en la máquina virtual, aislando el sistema anfitrión de posibles daños o vulnerabilidades.
- **Amplio ecosistema:** Frameworks como Spring, JavaEE (Jakarta EE), herramientas de desarrollo (Eclipse, IntelliJ, NetBeans) y una comunidad muy activa.

■ comentario de una linea

comentario de una linea

```
1 | // Esto es un comentario de una linea
```

■ comentarios de múltiples lineas

comentario múltiple lineas

```
1 | /** Este estilo es JavaDoc  
2 |  * También funciona como comentarios  
3 |  * De varias lineas  
4 |  * o mas bien un bloque de codigo  
5 |  * @version 0.1  
6 |  * @autor UABC FCQI  
7 |  * permite tener marcadores para generar documentación utilizando  
8 |  * JavaDoc  
9 |  * /
```

Existen estándares para comentar código y poder generar documentación automática tales como:

- JavaDoc
- PyDoc

Java es un lenguaje que esta **fuertemente tipado** , esto quiere decir que al definir una variable se debe conocer de que tipo es.

Java tiene 8 tipos de datos primitivos

Type	Storage Requirement	Range (Inclusive)
int	4 bytes	-2,147,483,648 to 2,147,483,647 (just over 2 billion)
short	2 bytes	-32,768 to 32,767
long	8 bytes	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
byte	1 byte	-128 to 127

Type	Storage Requirement	Range
float	4 bytes	Approximately $\pm 3.40282347 \times 10^{38}$ (6–7 significant decimal digits)
double	8 bytes	Approximately $\pm 1.79769313486231570 \times 10^{308}$ (15 significant decimal digits)

char

Originalmente se usaba para describir caracteres individuales pero ya no es el caso, actualmente se utiliza para describir/utilizar **Unicode** ya que algunos se puede utiliza con 1 carácter y otros con 2, se representan con la comillas simple.

char

```
1 char a = 'A';  
2 //error  
3 char b = "B"; //← esto no es un char, es un String
```

boolean

estos tipo de dato solamente contiene dos valores **true** y **false**
se utilizan para evaluar condiciones lógicas

En Java el 0 no se evalúa como falso como en el lenguaje C/C++

boolean

```
1 | boolean x=false;  
2 | /* no quiere decir falso , quiere decir que X es igual a 0 y no compila  
3 | if (x == 0)  
4 | {  
5 |     ...  
6 | }*/  
7 | //debería de ser  
8 | if (x==false)  
9 | {  
10 |     ...  
11 | }
```

variables

```
1 //una variable no puede empezar con numero o caracter especial ni espacios
2 double salario;
3 float calificacion= 0.f;
4 int semestre=5;
5 boolean fin;
6 long poblacion_mundial
7 char letra1;
8 //constantes en java se usa final
9 final double PI = 3.1415;
10 //la palabra const esta reservada como parte del lenguaje pero no se usa para nada
```

Cuando una variable necesite almacenar un conjunto de valores restringidos se pueden utilizar enumeradores , por ejemplo el tamaño de una pizza

Enumeradores

```
1 enum pizza_tam = {  
2     EXTRA_GRANDE,  
3     GRANDE,  
4     MEDIANA,  
5     CHICA  
6 };  
7  
8 //se usaría de esta forma  
9 pizza_tam tamaño = pizza_tam.MEDIANA;
```

Java [Operadores]

Operators	Associativity
[] . () (method call)	Left to right
! ~ ++ -- + (unary) - (unary) () (cast) new	Right to left
* / %	Left to right
+ -	Left to right
<< >> >>>	Left to right
< <= > >= instanceof	Left to right
== !=	Left to right
&	Left to right
^	Left to right
	Left to right
&&	Left to right
	Left to right
?:	Right to left
= += -= *= /= %= &= = ^= <<= >>= >>>=	Right to left

Utilizando la clase de java **Math** tenemos acceso a múltiples funciones

- `Math.pow(valor)`
- `Math.sqrt(valor)`
- `Math.round(valor)`
- `Math.sin(valor)`
- `Math.cos(valor)`
- `Math.tan(valor)`
- `Math.exp(valor)`
- `Math.log(valor)`
- `Math.log10(valor)`
- `Math.PI` constante
- `Math.E` constante

Permite transformar de un tipo de dato a otro compatible.

Cast

```
1 float x = 9.997f;  
2 int z = (int)x; //casteo de flotante -> entero  
3 System.out.println(z); //imprime 9
```

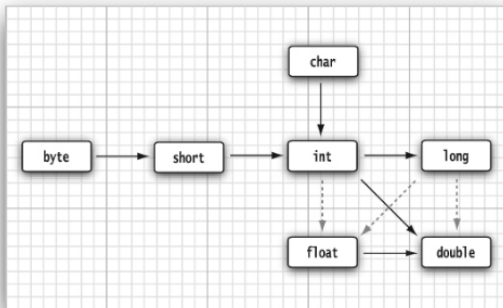


Figure 3.1 Legal conversions between numeric types

La líneas punteadas es donde hay una perdida de información

Utiliza la misma forma que lenguaje C para hacer incrementos o decrementos

```
1 | int i=7;
2 | int j=7;
3 | // esto da como resultado 16, ya que primero incrementa i
4 | int x = 2 * ++i;
5 | // esto da como resultado 14, ya que primero hace 2*7 y
6 | // despues incrementa j;
7 | int y = 2 * j++;
```

los operadores relacionales son los mismo que se manejan en cualquier otro lenguaje

- **==**, igual que
- **!=**, diferente que
- **>**, mayor que
- **>=**, mayor igual que
- **<**, menor que
- **<=**, menor igual que
- **&&**, and (lógico)
- **||** or (lógico)

Es un **if** de una sola linea

```
1 | int menor = (x < y ) ? x : y;  
2 | //equivalente a decir  
3 | if(x < y)  
4 | {  
5 |     menor = x;  
6 | }  
7 | else  
8 | {  
9 |     menor = y;  
10| }
```

- $>>$, Corrimientos ceros a la derecha (división entera)
- $<<$, Corrimientos ceros a la izquierda (multiplicación entera)
- $\wedge$, operación binaria XOR
- $\sim$, operación binaria NOT
- $\&$, operación binaria AND
- $|$, operación binaria OR

Conceptualmente con secuencias de caracteres UNICODE , java no tiene un tipo de dato String, por lo que para poder hacer uso se tiene que utilizar **La clase String e instanciar un objeto**

Los Strings son **Inmutables** quiere decir que no se pueden cambiar letras individuales ya que no es un arreglo de caracteres como en otros lenguajes, si no una clase concreta.

Para modificar letras individuales hay que recrear una nueva instancia o concatenar.

Ejemplo String

```
1 String mistring = ""; //cadena vacia
2 String saludo = "Hola"; //
3
4 //sub cadenas
5 String subcadena = saludo.substring(0,3); //"hol"
6
7 //concatenar
8 String concatenar = saludo + subcadena; // "holahol"
```

Strings tienen métodos útiles como:

- equals `if(saludo.equals("hola"))`
- equalsIgnoreCase `if(saludo.equalsIgnoreCase("HOLA"))`
- compareTo `if(saludo.compareTo("Hola")== 0)`
- length `int tam = saludo.length`
- charAt `saludo.charAt(2)`

```
1 | if(saludo.equalsIgnoreCase("Hola")) // si saludo es == a "hola"  
2 | {  
3 |     System.out.println("Bienvenido");  
4 | }
```

En el caso de String **no se usa el operador ==** para ver si dos cadenas de texto son iguales, ya que no son texto si no una instancia de una clase (objeto) y solo verificaría que si están almacenadas en el mismo espacio de memoria

Para mostrar información en la consola (stdin) se utiliza la librería estándar en específico **System.out** en donde podemos encontrar funciones como

- **print**, imprime un String en consola
- **println**, imprime un String en consola y agrega un salto de línea.

Ejemplo

```
1 | System.println("Esto es un print con salto de linea");  
2 | System.print("print numero "+2);
```

Se puede formatear el texto que queremos mostrar en la consola, al estilo de C utilizando la clase **String** y su método **format**

```
1|System.out.println(String.format("pi es %5.2f y euler en exponencial %.4e", Math.PI, Math.E));
```

Los caracteres de escape y sus significados para formatiar un texto.

Conversion Character	Type	Example
d	Decimal integer	159
x or X	Hexadecimal integer. For more control over hexadecimal formatting, use the <code>HexFormat</code> class.	9f
o	Octal integer	237
f or F	Fixed-point floating-point	15.9
e or E	Exponential floating-point	1.59e+01
g or G	General floating-point (the shorter of e and f)	—
a or A	Hexadecimal floating-point	0x1.fccdp3
s or S	String	Hello
c or C	Character	H
b or B	boolean	true
h or H	Hash code	42628b2
tx or Tx	Legacy date and time formatting. Use the <code>java.time</code> classes instead—see Chapter 6 of Volume II.	—
%	The percent symbol	%
n	The platform-dependent line separator	—

Flag	Purpose	Example
+	Prints sign for positive and negative numbers.	+3333.33
space	Adds a space before positive numbers.	3333.33
0	Adds leading zeroes.	003333.33
-	Left-justifies field.	3333.33
(	Encloses negative numbers in parentheses.	(3333.33)
,	Adds group separators.	3,333.33
# (for f format)	Always includes a decimal point.	3,333.
# (for x or o format)	Adds 0x or 0 prefix.	0xcafe
\$	Specifies the index of the argument to be formatted; for example, <code>%1\$d %1\$s</code> prints the first argument in decimal and hexadecimal.	159 9f
<	Formats the same value as the previous specification; for example, <code>%d %<s</code> prints the same number in decimal and hexadecimal.	159 9f

Para poder capturar la información del usuario en Java tenemos que usar la clase **Scanner** el cual adhiere el **System.in** y podemos guardarlo en una variable.

Esta clase contiene métodos como:

- **next()**, adquiere el texto del stdin **sin espacios**
- **nextLine()** adquiere el texto del stdin **incluyendo espacios.**
- **nextInt(),nextFloat(),nextDouble()** adquiere el texto del stdin y lo convierte al dato deseado

para mas información de los métodos que contiene la clase, revisar la documentación

<https://devdocs.io/openjdk-21/java.base/java/util/scanner>

Input

```
1 public class Capturar
2 {
3     public static void main(String[] args)
4     {
5         //capturar
6         Scanner input = new Scanner(System.in);
7
8         System.println("Cual es tu nombre?");
9         // puede contener espacios por eso nextLine
10        String nombre = input.nextLine();
11        //String nombre = input.next(); //si no va contener espacios
12        System.println("En que semestre vas?");
13        int semestre = input.nextInt();
14        System.println("Tu nombre es " + nombre + " y vas en el semestre " + semestre );
15    }
16 }
```

En java para abrir un bloque de código mejor conocido como ámbito o en ingles **scope**, se tienen que poner las llaves `{}` que lo simboliza.

```
1 public class Main
2 {
3     public static main(String[] args)
4     {
5         int x=0;
6         { //nuevo bloque de código
7             //existe x en este bloque
8             int y=20;
9         } // fin del bloque de código
10        // 'y' no existe en este bloque
11    }
12 }
```

if-else

```
1 // #1
2 if( condicional )
3 {
4     // si se cumple la condicional
5 }
6 // #2
7 if( condicional )
8 {
9     // si se cumple la condicional
10 }
11 else
12 {
13     // si no se cumple la condicional
14 }
15 // #3
16 if( condicional1 )
17 {
18     // si se cumple
19 }
20 else if( condicional2 )
21 {
22     // no se cumple pero verifica otra condicional
23 }
24 // ... muchos else if
25 else
26 {
27     // no se cumple ninguna condicional
28 }
```

Es diferente al que tradicionalmente se maneja en C, este puede tomar como valor, un String, enteros, o enumeradores y puede regresar valores, sin embargo se puede utilizar como se usa en C.

por ejemplo como si fuera C

```
1 Scanner in = new Scanner(System.in);
2 System.out.print("Selecciona una opcion: (1, 2,3,4) ");
3 int opcion = in.nextInt();
4 switch (opcion)
5 {
6     case 1:
7     case 4:
8         // código
9         break;
10    case 2:
11    case 3:
12        // código
13        break;
14    default:
15        System.out.println("valor incorrecto");
16 }
```

Por ejemplo el modo java

```
1 Scanner in = new Scanner(System.in);  
2 System.out.print("Select an option (1, 2,3, 4) ");  
3 int choice = in.nextInt();  
4 switch (choice)  
5 {  
6     case 1,4->  
7         //codigo  
8     case 2,3->  
9         //codigo  
10    default ->  
11        System.out.println("valor incorrecto");  
12 }
```

Otro ejemplo el modo java

```
1 enum Size { SMALL, MEDIUM, LARGE, EXTRA_LARGE }; . . .
2 Size itemSize = LARGE;
3 String label = switch (itemSize)
4 {
5     case SMALL -> "S"; // no need to use Size.SMALL
6     case MEDIUM -> "M";
7     case LARGE -> "L";
8     case EXTRA_LARGE -> "XL";
9 };
10
11 int numLetters = switch (seasonName)
12 {
13     case "Spring", "Summer", "Winter" -> 6;
14     case "Fall" -> 4;
15     default -> -1;
16 };
17
18 String seasonName = switch (seasonCode)
19 {
20     case 0 -> "Spring";
21     case 1 -> "Summer";
22     case 2 -> "Fall";
23     case 3 -> "Winter";
24     default -> "???";
25 };
```


Opera de la misma manera que lenguaje C

while sintaxis

```
1 | while (condición)
2 | {
3 |     //repetir código
4 | }
```

Funciona igual que C

do-while sintaxis

```
1 // garantiza 1 repetición
2 do
3 {
4     //repetir código
5 } while (condición);
```

Funciona igual que el lenguaje C

```
1 | // (inicio;condicion_termino;incremento)
2 | for(int i=0; i<10;++i)
3 | {
4 |     //repeticion
5 | }
```

Es un ciclo para iteradores (un arreglo o una colección), en donde cada iteración agarramos un elemento del arreglo o colección, por lo que no es necesario saber la cantidad de elementos.

```
1 | int num[]={1,2,3};  
2 | for(int n : num)  
3 | {  
4 |     System.out.println(n); //imprime 1, 2, 3  
5 | }
```

Java ofrece la palabra reservada para interrumpir a los ciclos.
break interrumpe el ciclo más próximo.

break

```
1 for (int i=0; i<100; i++)  
2 {  
3     for (int j =0; j<100; ++j)  
4     {  
5         if ( i+j == 300)  
6         {  
7             break; //rompe el ciclo interior  
8         }  
9     }  
10 }
```

break también puede actuar como si fuera **goto** de ensamblador y salta a una etiqueta

```
1 salir:
2 for(int i=0;i<100;i++)
3 {
4     for(int j =0;j<100;++j)
5     {
6         if( i+j == 300)
7         {
8             break salir;
9             // rompe ambos ciclos ya que en realidad rompio
10            // el bloque de codigo de la etiqueta salir
11        }
12    }
13 }
```

la palabra reservada **continue** actua para saltarse iteraciones pero no interrumpe ciclos.

```
1 | for(int j =0;j<100;++j)
2 | {
3 |     if( (j+1)&1 == 1)
4 |     {
5 |         continue; //se salta todos los numeros pares
6 |     }
7 | }
```

Un arreglo en java, almacena una colección de datos del mismo tipo, en donde puedes acceder a cada dato individual con un index

```
1 // declaracion
2 int [] arreglo;
3
4 //inicializacion
5 arreglo = new int [10];
6 float [] otro_arreglo = new float [3];
7 int [] arr = {1,2,3};
```


una vez definidos la cantidad de elementos que va utilizar, no se puede cambiar, por lo que si trata de ingresar indexes que están mas altos que la cantidad el compilador marcara una excepción

Como los arreglos es una variable que apunta a una referencia de un espacio en memoria, no se puede copiar directamente **debems de hacer uso de una clase llamada Arrays** que se encuentra en **java.util.Arrays**

copiar arreglos

```
1 import java.util.Arrays;
2 public class Main
3 {
4     public static void main(String[] args)
5     {
6         int a[3]= {1,2,3};
7         int []b = a;
8         a[2]= 100;
9         //imprime 1,2,100
10        System.out.println (Arrays.toString(b));
11        b = Arrays.copyOf(a,a.length);
12        a[2]=10000;
13        //imprime 1,2,100
14        System.out.println (Arrays.toString(b));
15    }
16 }
```

**** más adelante se profundizará en arreglos y en funciones (métodos) ****

En java los parámetros de una función son pase por valor en otras palabras copias, no es posible cambiar el valor de la variable, pero lo que guarda si es una referencia como un arreglo o un objeto si se puede modificar.

para estos ejemplos se utilizará el modificador static (se verá más adelante el porque.)

párametros

```
1 import java.util.Arrays;
2 public class Main
3 {
4     public static void main(String[] args)
5     {
6         int[] arr = {1,2,3};
7         mod(arr);
8         // como es por copia, la variable arr
9         // no se modifica en la función mod
10        System.out.println(Arrays.toString(arr));
11        mod2(arr);
12        // el elemento 2 de arr cambio a 10
13        // ya que la variable arr tiene referencias
14        // al contenido en este caso a los elementos
15        // por lo que se puede editar
16        System.out.println(Arrays.toString(arr));
17    }
18    public static void mod(int[] a)
19    {
20        a = new int[3];
21        a[0]=10;
22        a[1]=20;
23        a[2]=30;
24    }
25    public static void mod2(int[] a)
26    {
27        a[2]=10;
28    }
29 }
```

Se quiere hacer un sistema de calificaciones de un salón de clases, en donde te muestre la lista de los alumnos, las calificaciones parciales y si el alumno aprobó o no la materia.

(por ahorita todo en el main y estático para no entrar a orientado a objetos)

Ejemplo completo

```

1 import java.util.Random;
2 public class SistemaCalificacion
3 {
4     public static void main(String[] args)
5     {
6         //40 Strings nulos
7         String alumnos[] = new String[40];
8         //40x5 floats en 0
9         float parciales [][] = new float[40][5];
10
11         Random rand = new Random();
12         //llenar al azar el nombre
13         String nombres_temp[] = {"AMAR", "PEDRO", "JUAN", "ROSARIO", "JESICA", "CARLOS", "ANNA", "CARLA"};
14         String apellido_temp[] = {"PEREZ", "GONZALES", "HERNANDEZ", "AVALOS", "BARBA", "QUINO", "FLORES", "CORDOBA"};
15
16         for(int i=0; i<40; i++)
17         {
18             int id = rand.nextInt(7);
19             alumnos[i] = nombres_temp[id] + " " + apellido_temp[id];
20             for(int j=0; j<5; j++)
21             {
22                 // el random es de 0.0 a 1.0
23                 parciales[i][j] = (0.4+rand.nextFloat())*100.f;
24             }
25         }
26         //imprimir la tabla de aprobo no
27         System.out.println(String.format("| %-18s | %-5s | %-5s | %-5s | %-5s | %-5s | %-5s | %-10s | ", "NOMBRE", "P1", "P2", "P3", "P4", "P5", "TOTAL", "APROBO"));
28         for(int a=0; a<40; a++)
29         {
30             float total = SumaParciales(parciales[a]);
31             System.out.println(String.format("| %-18s | %-2.3f | %-2.3f | %-2.3f | %-2.3f | %-2.3f | %-2.3f | %-10s | %-10s | ", alumnos[a],
32             parciales[a][0], parciales[a][1], parciales[a][2], parciales[a][3], parciales[a][4],
33             total, Aprobo(total)));
34         }
35     }
36     public static float SumaParciales(float[] p)
37     {
38         float s=0;
39         for(int i=0; i<p.length; i++)
40         {
41             s+=p[i];
42         }
43         return s/p.length;
44     }
45     public static boolean Aprobo(float c)
46     {
47         return c>=60;
48     }
49 }

```